2025 Fall
week 4: C# in Unity1

게임프로그래밍 (Game Programming)

CSE at SeoulTech

Contents

- Week4: C# in Unity 1
- 4.1 상속과 재사용
- 4.2 컴포넌트 패턴 : 조립하는 게임 세상
- 4.3 유니티 에디터에서의 컴포넌트
- 4.4 메시지와 브로드캐스팅

4.1 상속과 재사용(1)

- 게임 엔진은 이미 완성된 기반 코드를 제공
- 게임 엔진의 코드를 '재사용'하므로 생산성 향상
- 코드를 재사용하는 전통적 방법인 '상속'
  - 부모 (parent) 클래스 - 기초를 제공하는 클래스
  - 자식 (child) 클래스 - 부모 클래스를 상속해서 확장한 클래스



▶ 상속

```
class Monster {

    // 몬스터에 관한
    // 변수와 메서드들...

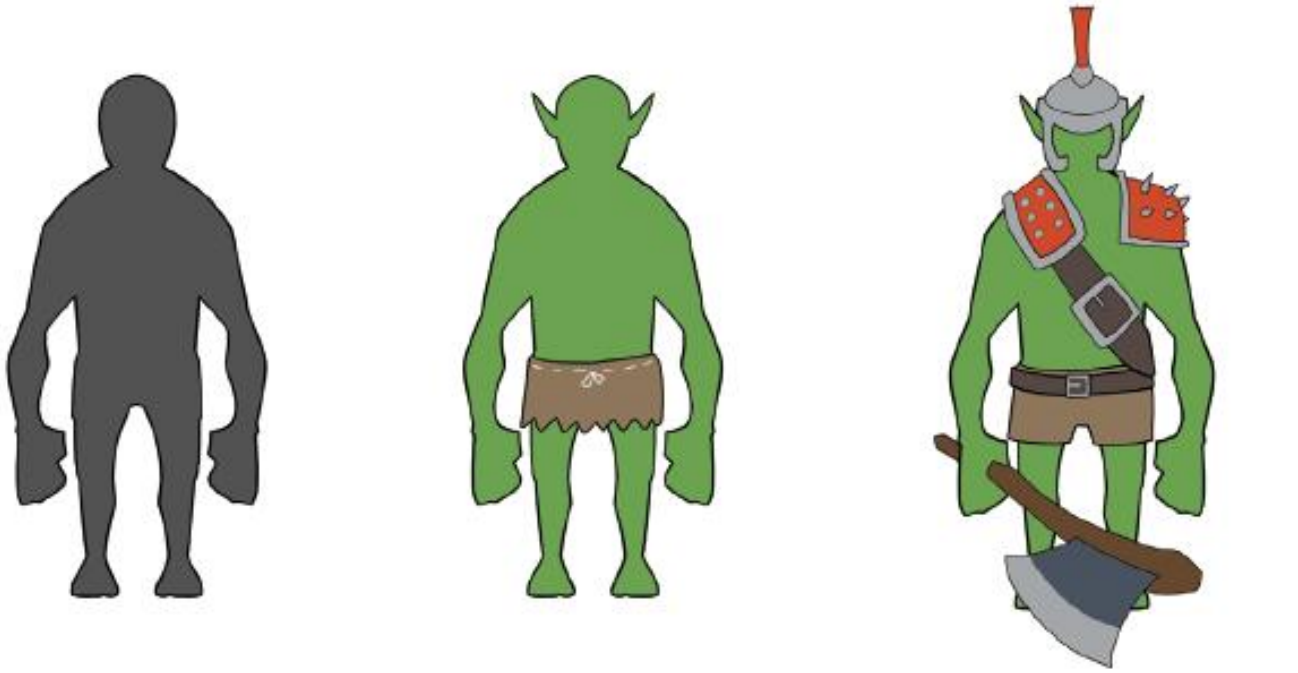
}
```

▶ 클래스의 모습

4.1 상속과 재사용(2)

4.1.1 상속으로 몬스터 만들기

- 개발자가 상속으로 게임 속 몬스터를 구현



class **Monster**　　class **Orc** : Monster　　class **OrcChieftan** : Orc

▶ 구현할 몬스터 클래스들

4.1 상속과 재사용(3)

[과정 01] 몬스터(Monster) 클래스 구현



class **Monster**

- 인공지능 기능
- 애니메이션 기능
- 공격과 방어 기능
- 물리 기능
- 기타 필수 기능

[과정 02] 몬스터를 기반으로 오크(Orc) 클래스 구현



class **Orc** : Monster

- 초록색 피부
- 오크의 애니메이션
- 오크의 스킬
- 그외 오크의 고유 기능

[과정 03] 오크를 기반으로 오크 대장 (Orc Chieftan) 클래스 구현



class **OrcChieftan** : Orc

- 대장 모자
- 새로운 무기와 강력한 스킬
- 그외 오크 대장의 고유 기능

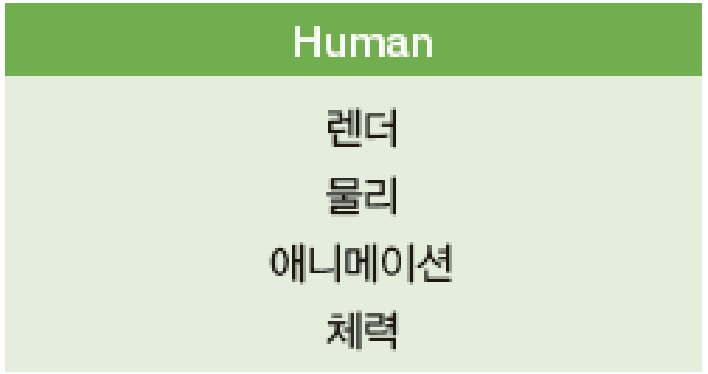
4.1 상속과 재사용(4)

4.1.2 상속의 한계

- 부모 클래스를 상속해 자식 클래스의 기초 구현을 대신할 수 있음
- 하지만 상속에만 의존하면 오히려 코드를 재사용하기 힘들 수 있음

[과정 01] 최상위 부모 클래스인 사람(Human) 클래스 구현

- 모습을 그려주는 렌더 기능
- 물리 기능
- 애니메이션 기능
- 체력 기능
- 기타 필수 기능



▶ Human 클래스

4.1 상속과 재사용(5)

[과정 02] 플레이어(Player) 클래스 구현

Human 클래스의 기능 위에 새로 추가한 Player의 기능

- 조작 기능
- 공격 기능
- 직업 기능
- 그외 필수 기능



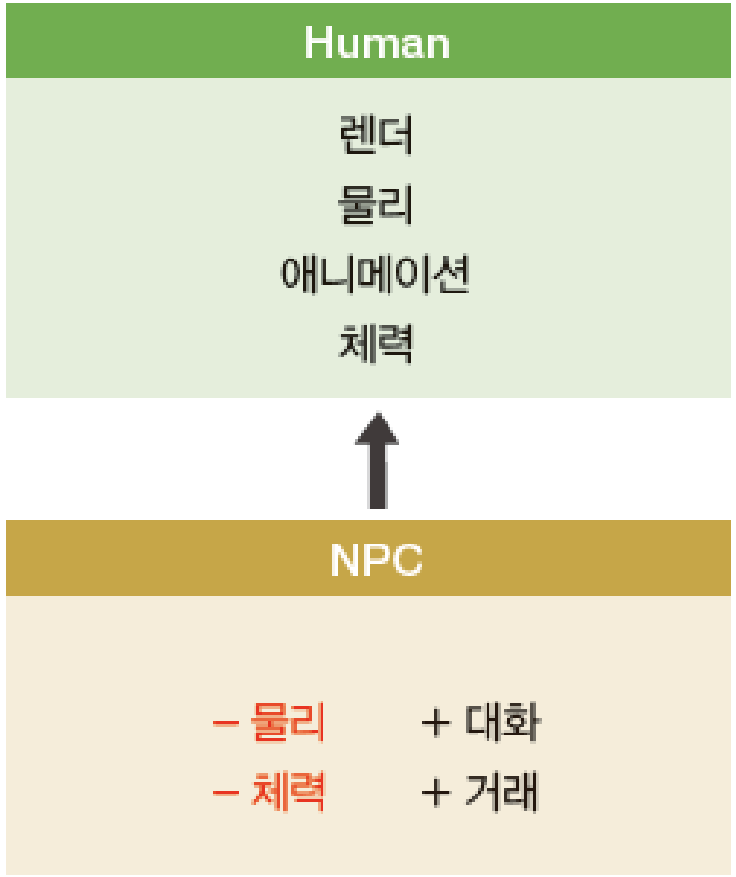
▶ Human 클래스

4.1 상속과 재사용(6)

[과정 03] NPC 클래스 구현

제거하고 추가한 기능

- 물리 기능 삭제
- 체력 기능 삭제
- 대화 기능 추가
- 거래 기능 추가



▶ Human을 상속한 NPC

4.1 상속과 재사용(7)

[과정 04] 몬스터(Monster) 클래스 구현

수정하고 추가한 기능

- 기존 애니메이션을 몬스터 애니메이션으로 수정
- 기존 물리를 몬스터 물리로 수정
- 몬스터에 어울리지 않는 Human의 기능 삭제
- 인공지능 기능 추가
- 공격 기능 추가



▶ Human을 상속한 Monster

4.1 상속과 재사용(8)

4.1.3 결론

- 상속에만 의존하면 오히려 기존 코드를 재사용하기 힘든 경우가 생길 수 있음
- 부모 클래스에는 자식 클래스에 공통적으로 필요한 기능을 구현
- 부모 클래스의 기존 기능이 나중에 구현한 자식 클래스의 기능과 오히려 충돌할 수도 있음
- 상속에만 의존하여 게임을 개발할 때 생기는 문제점
  - 오히려 코드를 재사용하기 힘든 경우가 발생
  - 기획자가 새로운 오브젝트를 만들려면 프로그래머에게 의존
- 이러한 문제를 해결하려면 컴포넌트 패턴을 사용

4.2 컴포넌트 패턴 : 조립하는 게임 세상(1)

4.2.1 컴포넌트로 동물 만들기

- 유니티는 게임 오브젝트를 컴포넌트 패턴을 사용
- 컴포넌트 패턴 혹은 컴포지션 패턴이란 **미리 만들어진 부품을 조립**하여 완성된 오브젝트를 만드는 방식
  - 기획자와 프로그래머가 컴포넌트로 게임 속 동물을 만든다고 가정

[과정 01] 컴포넌트를 미리 여러 개 만들기



[과정 02] 빈 게임 오브젝트 생성하기



4.2 컴포넌트 패턴 : 조립하는 게임 세상(2)

[과정 03] 코뿔소 게임 오브젝트 완성하기



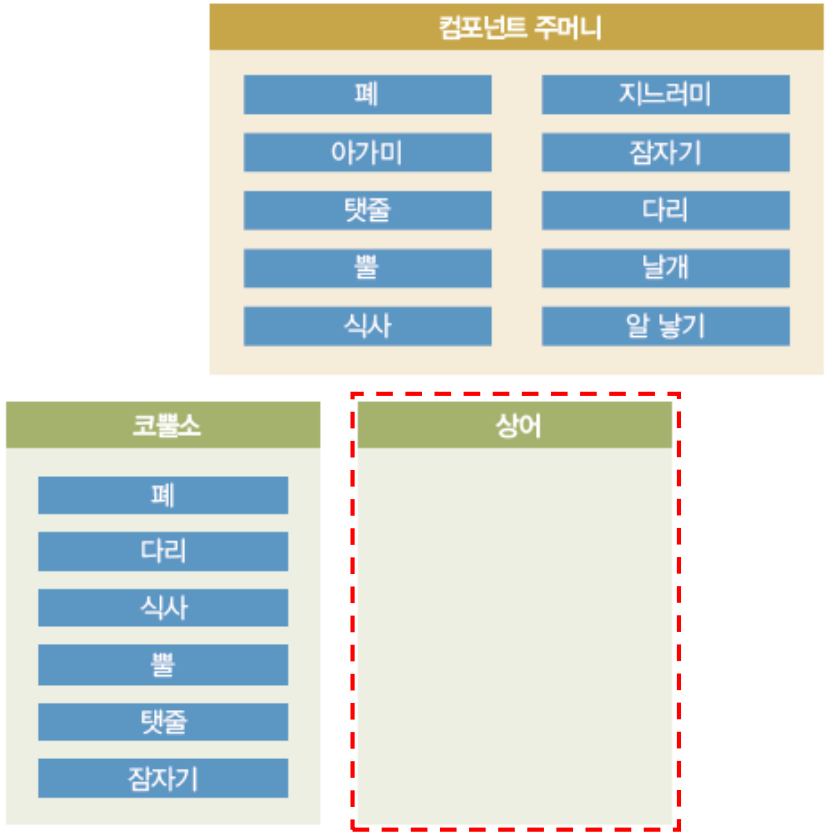
▶ 코뿔소에 컴포넌트 추가



▶ 완성된 코뿔소 게임 오브젝트

4.2 컴포넌트 패턴 : 조립하는 게임 세상(3)

[과정 04] 상어와 독수리 게임 오브젝트 생성하기



▶ 상어 게임 오브젝트



▶ 상어 게임 오브젝트에 컴포넌트 추가

4.2 컴포넌트 패턴 : 조립하는 게임 세상(4)

독수리라는 이름의 빈 게임 오브젝트를 생성하고 컴포넌트를 추가하여 독수리를 완성



▶ 독수리 게임 오브젝트 완성

4.2 컴포넌트 패턴 : 조립하는 게임 세상(5)

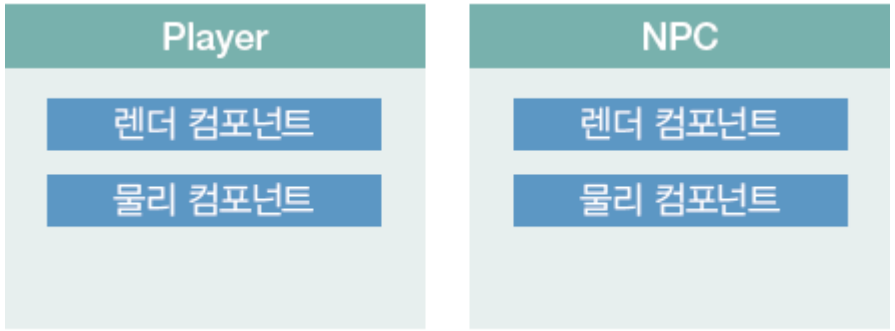
4.2.2 게임 오브젝트와 컴포넌트의 특징

- 컴포넌트 방식은 미리 만들어진 컴포넌트를 빈 껍데기인 게임 오브젝트에 조립하는 방식
- 컴포넌트 방식 세 가지 장점
  - 유연한 재사용이 가능
  - 기획자의 프로그래머 의존도가 낮아짐
  - 독립성 덕분에 기능 추가와 삭제가 쉬움

4.2 컴포넌트 패턴 : 조립하는 게임 세상(6)

4.2.3 컴포넌트의 독립성

- 컴포넌트 패턴 장점의 두 가지 특징
  - 게임 오브젝트는 단순한 빈 껍데기
  - 컴포넌트는 스스로 동작하는 독립적인 부품



▶ 플레이어와 NPC

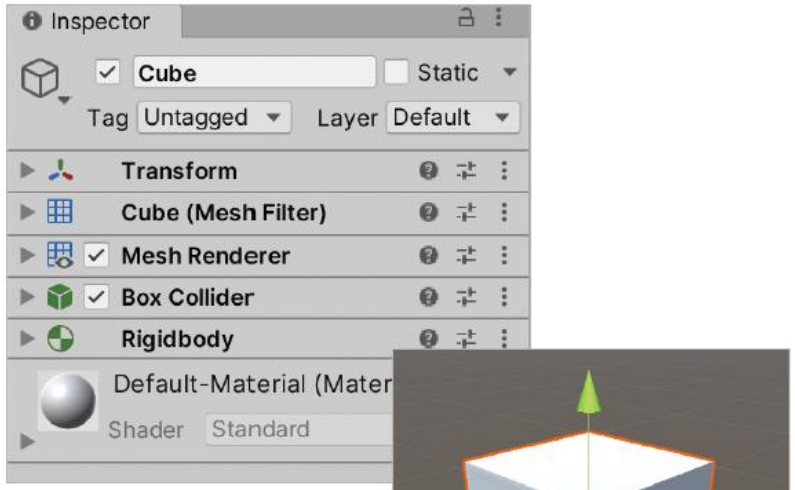


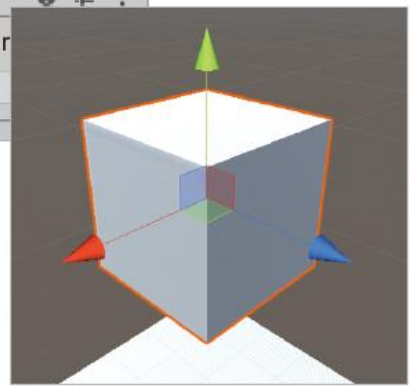
▶ 수정된 플레이어와 NPC

❖ **각자 새로운 기능을 가진 컴포넌트를 추가했지만 기존 컴포넌트를 수정할 필요는 없음**

4.3 유니티 에디터에서의 컴포넌트(1)

- Cube 게임 오브젝트의 컴포넌트
  - 트랜스폼(Transform) : 게임 오브젝트의 위치와 크기, 회전을 지정
  - 메시 필터(Mesh Filter) : 오브젝트의 외곽선을 지정
  - 메시 렌더러(Mesh Renderer) : 메시를 따라 색을 채워 그래픽 외형 그리기
  - 박스 콜라이더(Box Collider) : 다른 물체가 부딪칠 수 있는 물리적인 표면 만들기
  - 리지드바디(Rigidbody) : 게임 오브젝트의 물리 엔진 통제

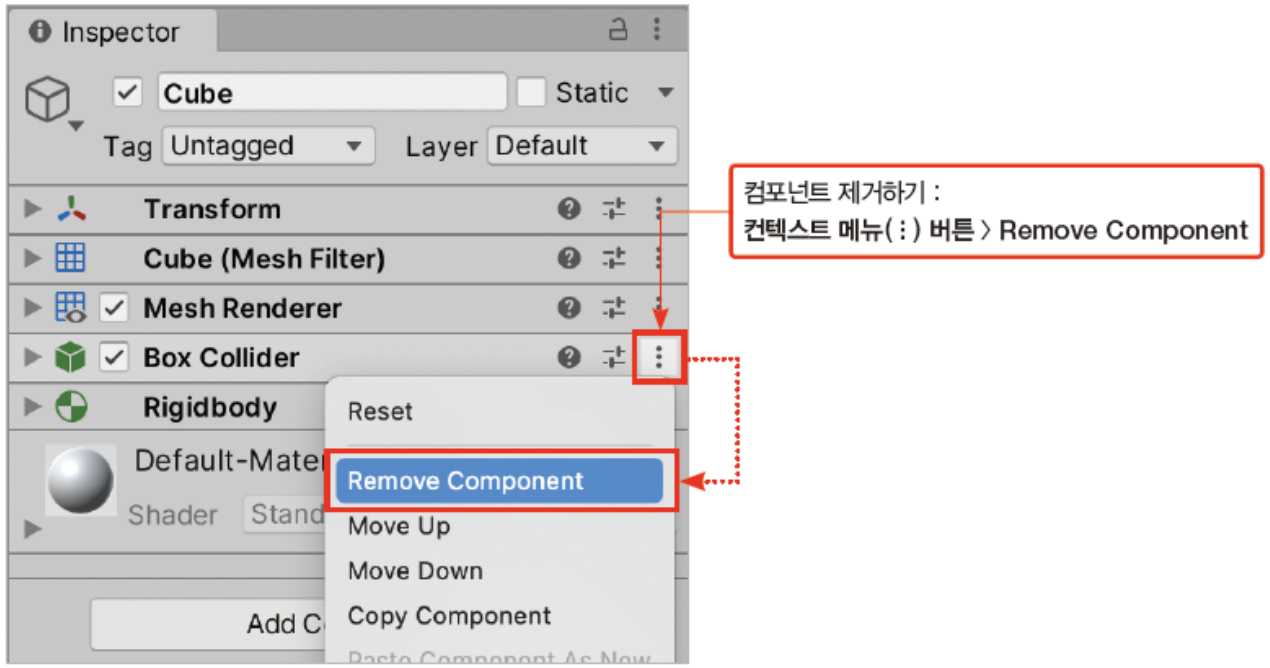




▶ Cube에 추가된 컴포넌트들

4.3 유니티 에디터에서의 컴포넌트(2)

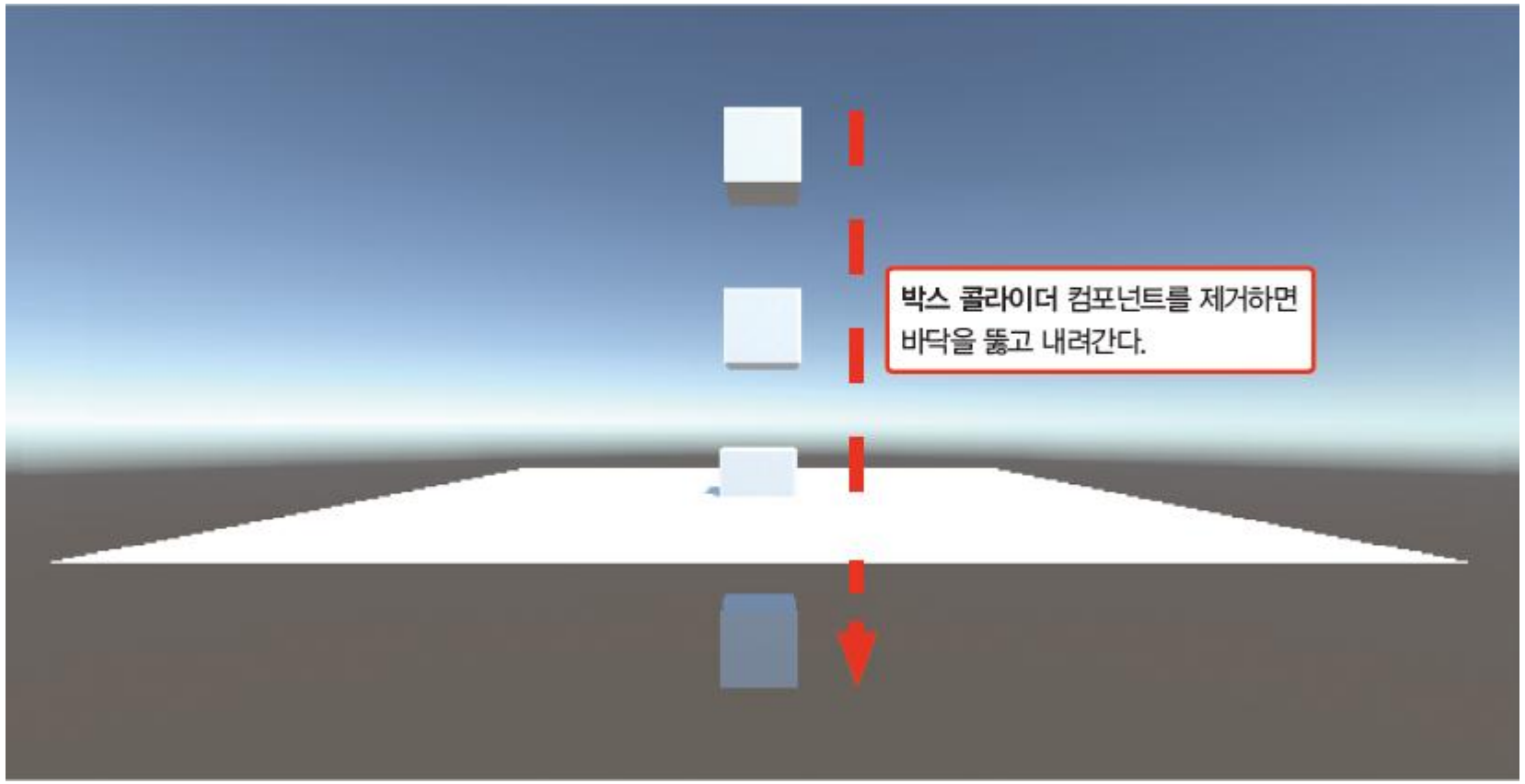
- 컴포넌트를 추가
  - 인스펙터 창에서 Add Component 버튼을 클릭
- 기존 컴포넌트를 제거
  - 컴포넌트를 마우스 오른쪽 버튼으로 클릭
  - 또는, 컨텍스트 메뉴(⋮) 버튼을 클릭한 다음 Remove Component를 선택



▶ 컴포넌트를 제거하는 방법

4.3 유니티 에디터에서의 컴포넌트(3)

- 만일 week2의 Cube 게임 오브젝트에서 박스 콜라이더 컴포넌트를 제거하면?
  - Cube 게임 오브젝트의 물리적인 표면이 사라짐
  - 따라서 큐브는 다른 물체와 충돌하지 않고 그대로 뚫고 지나가 영원히 떨어지게 됨



▶ 박스 콜라이더를 제거한 큐브

4.4 메시지와 브로드캐스팅(1)

4.4.1 MonoBehaviour는 무엇인가

- 브로드캐스팅 -컴포넌트 구조에서는 '전체 방송'을 이용해 컴포넌트의 특정 기능을 간접적으로 실행
- 유니티의 **모든 컴포넌트는 MonoBehaviour 클래스를 상속**
  - MonoBehaviour 클래스는 유니티에서 미리 만들어 제공하는 클래스이며 컴포넌트에 필요한 기본 기능을 제공
- MonoBehaviour를 상속해서 만든 컴포넌트는 유니티의 제어를 받음



▶ 유니티 컴포넌트의 구조

4.4 메시지와 브로드캐스팅(2)

4.4.2 메시지 기반 방식

- 컴포넌트 패턴에서 컴포넌트들은 서로 관심이 없음
- 유니티는 컴포넌트의 어떤 기능을 실행시키고 싶을 때 메시지를 날리는 방식을 사용
- 메시지 방식의 특징
  - 메시지를 보내는 쪽은 누가 받게 될지 신경 쓰지 않음
  - 메시지를 받은 쪽은 누가 보냈는지 신경 쓰지 않음
  - 메시지를 받았을 때 메시지에 명시된 기능을 가지고 있다면 실행, 관련 없다면 무시

4.4 메시지와 브로드캐스팅(3)

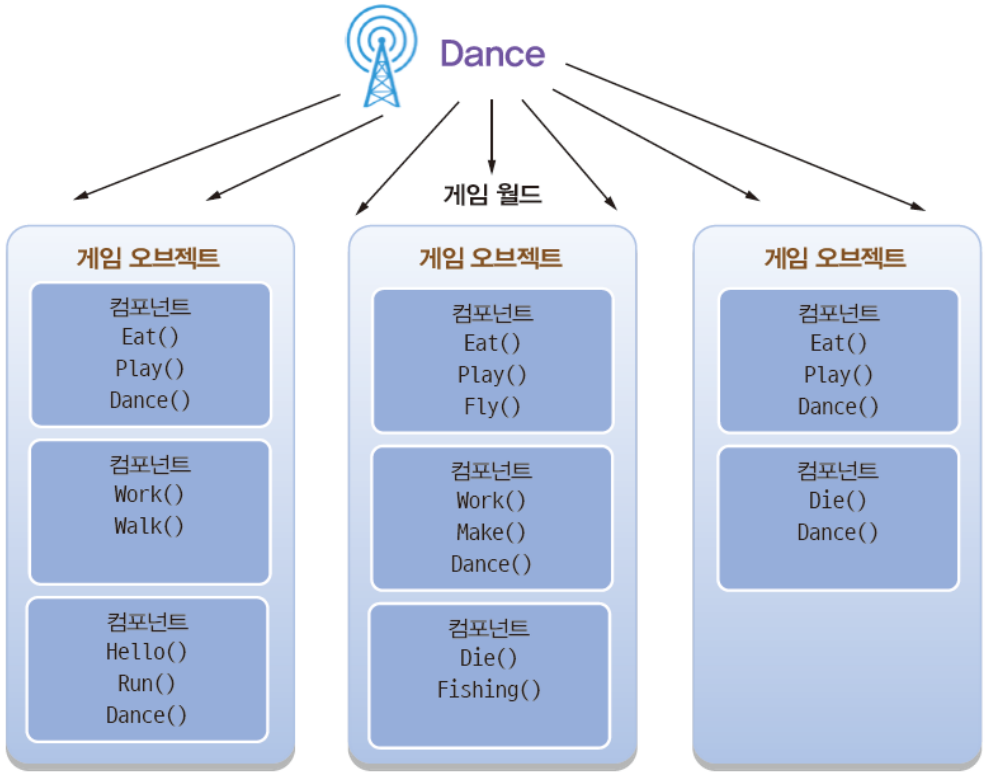
4.4.3 브로드캐스팅

- 유니티가 메시지로 원하는 기능을 동작시키는 방법
- 다음 그림처럼 게임 월드에 컴포넌트들이 추가된 3개의 게임 오브젝트가 있다고 가정
- 메서드 – Eat( ), Play( ), Dance( ) 등(컴포넌트가 가지고 있는 기능)
- 유니티가 컴포넌트들의 Dance( ) 기능을 일괄 실행

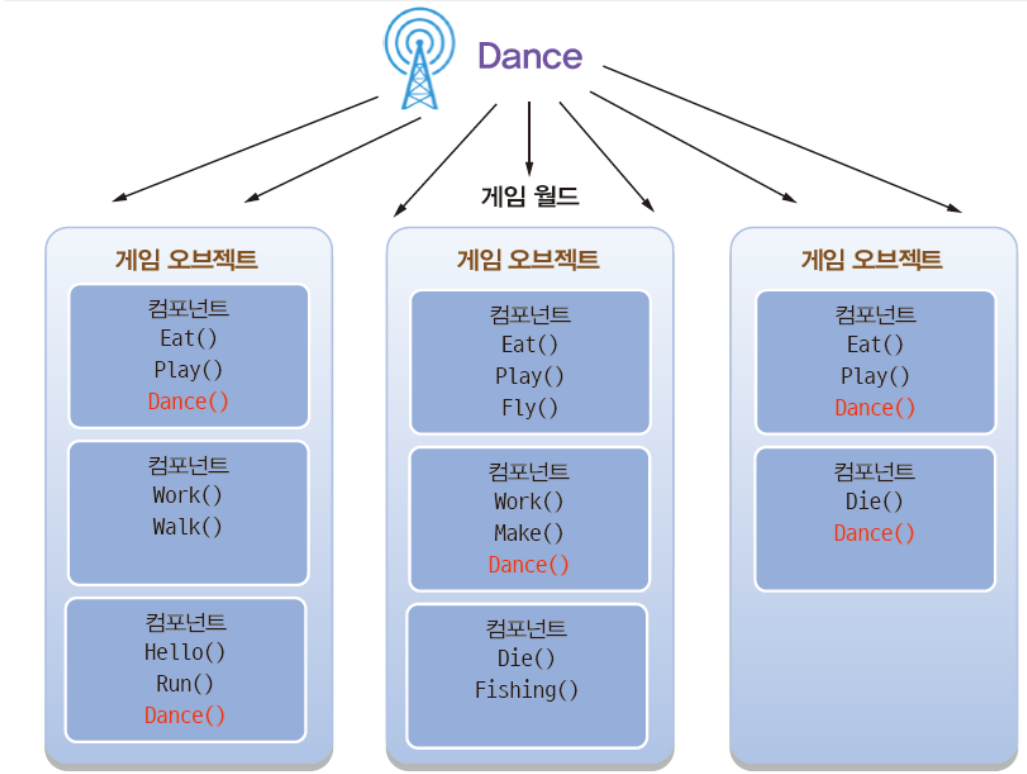


4.4 메시지와 브로드캐스팅(4)

- 메시지에 표시된 Dance를 가지고 있지 않다면 메시지를 무시
- 결과적으로 단 한 번 브로드케이팅으로 Dance( )를 가진 모든 컴포넌트가 Dance( )를 실행



▶ Dance를 브로드캐스팅



▶ 브로드캐스팅을 들은 컴포넌트들이 Dance( ) 실행

4.4 메시지와 브로드캐스팅(5)

4.4.4 유니티 이벤트 메서드

- 메시지와 브로드캐스팅은 Start( ), Update( ), OnTriggerEnter( ) 등의 '유니티 이벤트 메서드'가 동작하는 원리
- 유니티에는 이름 철자만 똑같이 구현해두면 메시지에 의해 자동으로 실행되는 메서드들이 있음
  - 이러한 메서드를 유니티 이벤트 함수 또는 유니티 이벤트 메서드라고 함